

CST2550 Group Coursework

Cinema Film Booking and Membership System

Adidas Group

Leader: Mohamed Aziz Hammami M01005543

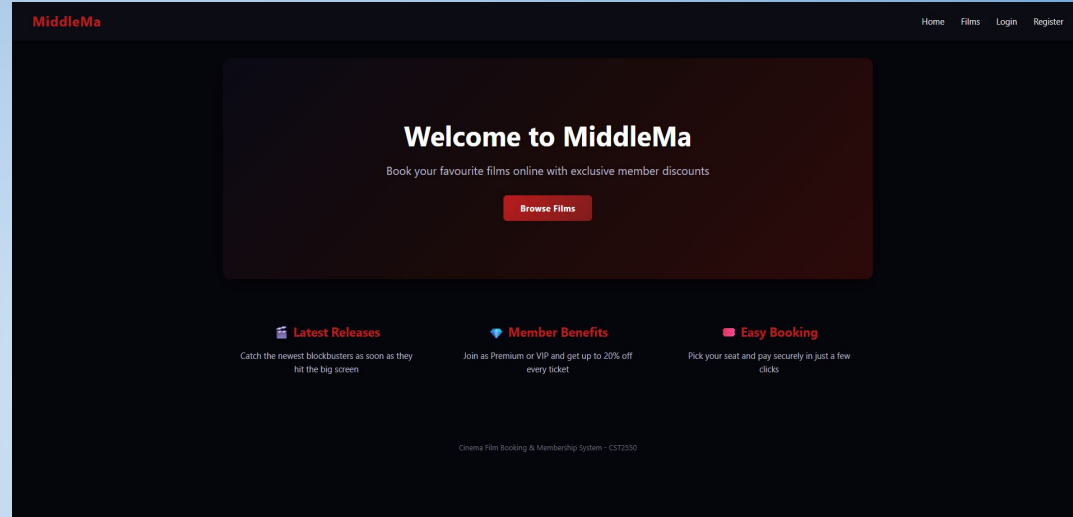
Secretary: Klejsti Carciu M01010867

Developer: Ali Al Tamimi M01042238

Developer: Deniz Ozer M01027961

Tester: Georgios Meltzanas M01031178

Web Based cinema booking platform



- Features:
- Browse and search films
- Register and login with accounts
- 3 membership tiers with discounts
- Buy tickets with card validation
- View purchased tickets anytime

Project Management

Date: 05/04/2026 from 14:00 to 17:30
Sprint Number: 5 | Polish and Submit

Name: Ali Al Tamimi
Attended - Yes
1. Frontend cleanup
2. Set up sql server localdb, ran the database script and reconnected the web app, data persists between sessions
3. Connection string wasn't right at first, had to add trustservercertificate

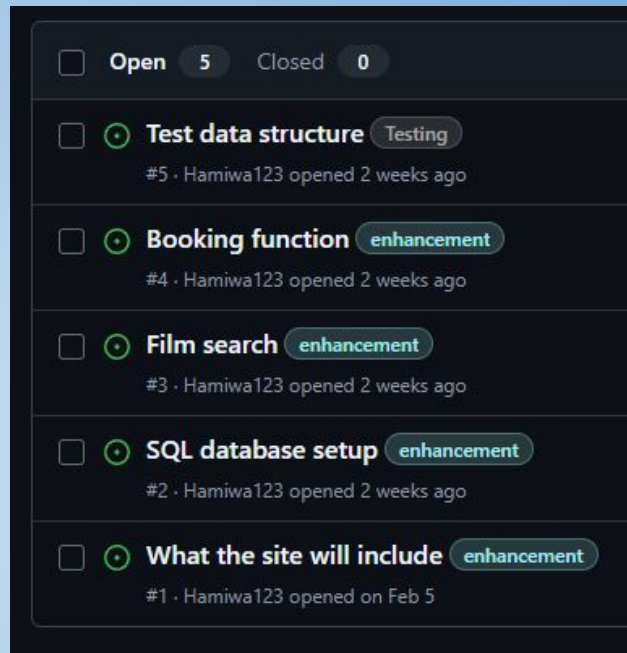
Name: Mohamed Aziz Hammami
Attended - Yes
1. Report pdf done
2. Checked the database script
3. None

Name: Klejsti Carciu
Attended - Yes
1. Final check of report
2. Checked the front end was working correctly
3. None

Name: Geogios Meltzanis
Attended - Yes
1. Tests all passing
2. Ran all tests with dotnet test and all passed
3. None

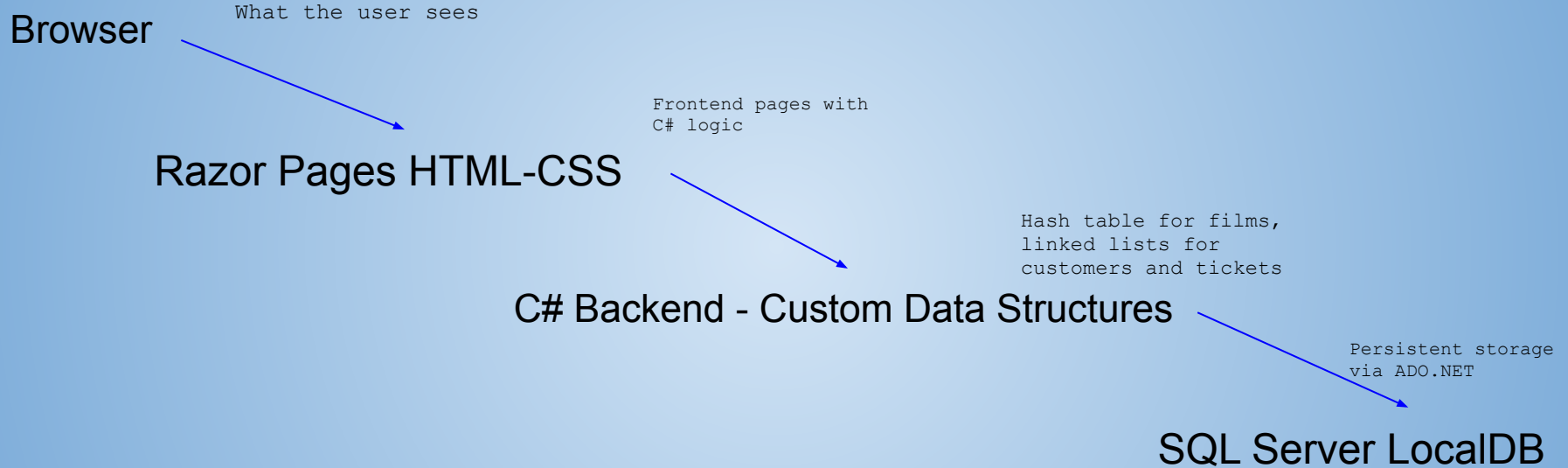
Name: Deniz Ozer
Attended - Yes
1. Code review
2. Helped ali with the database, added deletcustomer to the databasehelper
3. None

- Tracked through GitHub Issues
 - Held regular standup meetings which were documented to keep track of progress



- Organised work into sprints:
 - Setup
 - Design
 - Core Development
 - Testing
 - Polish

System Architecture



Data Structures

Custom Hash Table

```
public class CustomHashTable<TKey, TValue>
{
    9 references
    private HashNode<TKey, TValue>[] buckets;
    4 references
    private int size;
    5 references
    private int count;

    0 references
    public int Count { get { return count; } }

    1 reference
    public CustomHashTable(int tableSize)
    {
        size = tableSize;
        buckets = new HashNode<TKey, TValue>[size];
        count = 0;
    }

    // hash function - gets the index for a key
    4 references
    private int GetHashIndex(TKey key)
    {
        int hash = Math.Abs(key.GetHashCode());
        return hash % size;
    }

    // insert a key value pair into the table
    9 references
    public void Insert(TKey key, TValue value)
    {
        int index = GetHashIndex(key);

        // check if key already exists, if so update it
        HashNode<TKey, TValue> current = buckets[index];
        while (current != null)
        {
            if (current.Key.Equals(key))
            {
                current.Value = value;
                return;
            }
            current = current.Next;
        }

        // key doesn't exist so add new node at the front of the chain
        HashNode<TKey, TValue> newNode = new HashNode<TKey, TValue>(key, value);
        newNode.Next = buckets[index];
        buckets[index] = newNode;
        count++;
    }
}
```

- Used for films
- Separate chaining for collisions
- $O(1)$ average case lookup
- Fast retrieval when buying tickets

- Singly Linked List
- Dynamic size
- $O(n)$ search
- Groups as users register

Custom Linked List

```
public class CustomLinkedList<T>
{
    15 references
    private ListNode<T> head;
    7 references
    private int count;

    3 references
    public int Count { get { return count; } }

    4 references
    public CustomLinkedList()
    {
        head = null;
        count = 0;
    }

    // add to the front
    0 references
    public void InsertAtHead(T data)
    {
        ListNode<T> newNode = new ListNode<T>(data);
        newNode.Next = head;
        head = newNode;
        count++;
    }

    // add to the end
    7 references
    public void InsertAtTail(T data)
    {
        ListNode<T> newNode = new ListNode<T>(data);

        if (head == null)
        {
            head = newNode;
        }
        else
        {
            ListNode<T> current = head;
            while (current.Next != null)
            {
                current = current.Next;
            }
            current.Next = newNode;
        }
        count++;
    }
}
```

SQL Server LocalDB with 3 tables

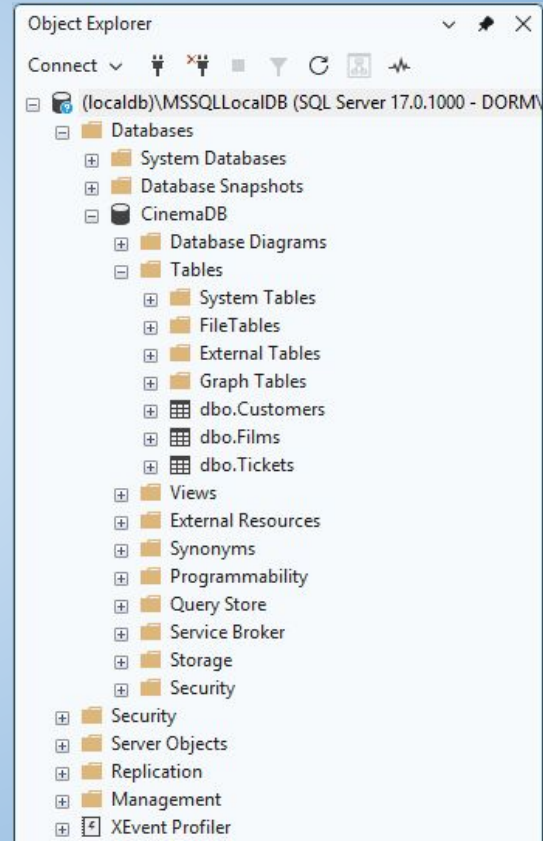
- Films- catalogue of available films
- Customers- account info
- Tickets- purchase records

Ado.Net used to read and write from C#

```
// load all films from db into the hash table
1 reference
public void LoadFilms(CustomHashTable<int, Film> filmTable)
{
    using (SqlConnection conn = new SqlConnection(connectionString))
    {
        conn.Open();
        string query = "SELECT * FROM Films";
        SqlCommand cmd = new SqlCommand(query, conn);
        SqlDataReader reader = cmd.ExecuteReader();

        while (reader.Read())
        {
            Film film = new Film();
            film.FilmId = (int)reader["FilmId"];
            film.Title = reader["Title"].ToString();
            film.Genre = reader["Genre"].ToString();
            film.Duration = (int)reader["Duration"];
            film.Rating = reader["Rating"].ToString();
            film.ShowTime = reader["ShowTime"].ToString();
            film.Price = (decimal)reader["Price"];
            film.AvailableSeats = (int)reader["AvailableSeats"];
            film.PosterUrl = reader["PosterUrl"].ToString();

            filmTable.Insert(film.FilmId, film);
        }
    }
}
```



Live Demonstration

See the system in action

Testing

33 Unit tests - All Passing

MSTest framework

- Film class: 5 tests
- Customer class: 4 tests
- Ticket class: 3 tests
- Hash table: 11 tests
- Linked Lists: 10 tests

```
[TestMethod]
0 references
public void TestFilmConstructor()
{
    Film film = new Film(1, "Inception", "Sci-Fi", 148, "12A", "18:30", 12.99m, 50);

    Assert.AreEqual(1, film.FilmId);
    Assert.AreEqual("Inception", film.Title);
    Assert.AreEqual("Sci-Fi", film.Genre);
    Assert.AreEqual(148, film.Duration);
    Assert.AreEqual("12A", film.Rating);
    Assert.AreEqual("18:30", film.ShowTime);
    Assert.AreEqual(12.99m, film.Price);
    Assert.AreEqual(50, film.AvailableSeats);
}
```

```
PS C:\Users\gmelt\OneDrive\Eλληνικά\GitHub\movie-project> cd src/CinemaSystem.Tests
PS C:\Users\gmelt\OneDrive\Eλληνικά\GitHub\movie-project\src\CinemaSystem.Tests> dotnet test
Restore complete (0.4s)
CinemaSystem net10.0 succeeded (0.3s) → C:\Users\gmelt\OneDrive\Eλληνικά\GitHub\movie-project\src\CinemaSystem\bin\Debug\net10.0\CinemaSystem.dll
CinemaSystem.Tests net10.0 succeeded (0.2s) → bin\Debug\net10.0\CinemaSystem.Tests.dll
CinemaSystem.Tests test net10.0 succeeded (0.7s)

Test summary: total: 33, failed: 0, succeeded: 33, skipped: 0, duration: 0.7s
Build succeeded in 2.1s
PS C:\Users\gmelt\OneDrive\Eλληνικά\GitHub\movie-project\src\CinemaSystem.Tests> |
```


Challenges and what we learned

- Started later than planned, had to catch up a lot
- Refactored from console app to web app mid development
- Learned how to coordinate in a team using git

Thank you!

GitHub repo: github.com/aliaaltamimm/movie-project